

Feature: Multi-tenant auth, with the org fix

From build-plan: feature 1

Branch: `feature/multi-tenant-auth-with-the-org-fix`

Status: verified, 2026-09-22. All six steps observed against the running app. The following turned out to be wrong in this spec and are recorded here rather than quietly corrected. No count is given on purpose: the list has been added to twice, and a stale number beside it misleads worse than no number.

- Step 6's `Done when` asks for "two organizations, **both** created only through the app". Only one could be: the database arrived holding the first repo's tenant, "The Latam Painters", adopted at step 1.2 rather than created here. The intent was that onboarding a tenant needs no database touch, and that is proved by the second one, "Ana's Hair & Co.". Deleting a real tenant to satisfy the letter of the gate would have been worse than recording the gap.
- Step 6 says to promote the admin role "through `drizzle-kit studio`". Done with a direct SQL update instead, which is the same act without a browser and can be shown in a transcript. The point of the instruction is that the promotion happens by hand and not through a request path, which holds.
- Step 1 lists `transpilePackages` in the Next config among its pieces, and it was deliberately dropped: once `packages/shared` compiled to JavaScript the setting did nothing, and config that does nothing is worse than none. `frontend/next.config.ts` is therefore untouched scaffold. The build log recorded this at the time; this preamble did not, which is the gap an independent review found as F-04.
- Step 1 describes a schema layout that never shipped. It lists `./db/schema` among the subpath exports and says to write the tables "under `src/db/schema/auth-schema/`", and Files / areas lists that directory plus `src/db/index.ts`. What exists is one file, `packages/shared/src/db/schema.ts`, with no barrel and no `db/index.ts`, and the third export is `./validation`, which step 1 never names. The one-file decision is argued at `schema.ts:19-23`: a barrel forces a relative import, and the two workspaces disagree about extensions. Recorded here as F-10, after F-04's repair fixed one gap of this kind and left this one.

`user.role` was then **proved** unsettable rather than asserted: a sign-in body carrying `role: "admin"` is refused with 400 `FIELD_NOT_ALLOWED`. `organization.plan` is safe too but fails differently, being silently replaced by its default on create. Both are documented at the declarations in `backend/src/lib/auth.ts`.

Goal

A second organization can be created through the app with no database touch, sign-in lands on that organization without a workaround, and a route behind the package gate refuses an organization whose rung does not include it.

This is the feature that decides where the trusted actor comes from. Every item from 2 to 26 reads its `organizationId` from whatever this feature establishes, so the boundary it draws is the one the product keeps.

Design reference

`prototypes/theme.css` and `prototypes/shell.css` hold the shared tokens: deep indigo accent, roomy light default, explicit light/dark toggle with no system setting. Port those tokens into the frontend before building any screen here.

There is no sign-in or create-organization mockup in `prototypes/`. Build both plainly from the ported tokens. Do not invent a richer treatment than the mockups that do exist.

In scope

- Better Auth on the Hono backend, with the `emailOTP`, `organization` and `admin` plugins.
- The Better Auth Drizzle schema in `packages/shared`, plus the first migration, plus the subpath-export and build wiring that makes the shared package consumable by both sides.
- `organization.plan`, text, default `agency`, server-set only.
- The auto-active-organization fix, as a session-create hook plus the read-time membership fallback.
- The plan-limits config with the one rung `agency`, and the gate middleware every module route will call.
- `GET /me`, the dashboard bootstrap route, behind auth and the gate.
- `/sign-in` and `/create-organization` on the frontend, plus the signed-in shell that reads `/me`.
- Credentialed CORS for the dashboard origin, kept separate from the public widget origins.
- Both signup doors closed. Not in the original scope: they arrived as the repair for F-05, the only P1 the independent review raised. Creating a business is restricted to the platform admin (`3030ef0`), and signing yourself up is disabled outright, a day later, once it was clear the door had no legitimate user. Together they mean every account in this product is one the agency deliberately created, which is what the project plan already said and the code did not.
- A local development database and `db:seed`. Also not in the original scope, and forced by the item twice over. The F-06 repair made a database buildable from empty for the first time, and closing signup made a freshly built one impossible to get into. `packages/shared/scripts/seed-dev.ts` creates one platform admin and one ordinary owner, each with a business, and refuses any database not on this machine or not named `*_dev`. Development moved off the Railway tunnel onto a local PostgreSQL 18 the same day.

Out of scope

- The admin area and the rungs above `agency` . Item 23. This feature ships the `admin` plugin and the gate, not the screens or the ladder.
- Real OTP email through Resend. Item 6 owns transactional email. This feature ships the send seam with a development transport.
- Any product table: `booking_link` , `availability_rule` , `contact` , `lead` . Items 2 and 4.
- The Hono RPC `AppType` seam. Feature 1's only frontend-to-backend calls go through the Better Auth client, which carries its own types. The RPC contract arrives with item 2's first public read route.
- Invitations, a second member in one organization, and any path at all that puts a client user inside a business. Build-plan item 3b owns all of it, added 2026-09-23 while reviewing this feature. The schema supports it because Better Auth ships the table.
- Google OAuth verification. Item 22.

Build loop

`stepReview` is `every` and `checkpointCommits` is `enabled` . Stop after each build step for review, and make a checkpoint commit on the feature branch once that step's `Done when` is observed. `/complete` makes the final feature commit and merges.

Build steps

- ☒ **1. Make `packages/shared` real, and put the Better Auth schema in it.** Replace the `main / types` stub with subpath exports (`./db` , `./db/schema` , `./config`), a `tsc` build emitting `dist/` , and a `drizzle.config.ts` . Declare `drizzle-orm` and `zod` as dependencies and `drizzle-kit` and `typescript` as devDependencies; all four are already in the tree, so this declares what is there rather than pulling anything new in. Write the Better Auth tables under `src/db/schema/auth-schema/` (`user` , `session` , `account` , `verification` , `organization` , `member` , `invitation`), ids text, timestamps `timestampz` , with the one added column `organization.plan` text not null default `'agency'` . Give `backend` a `prebuild` that runs `tsc -p ../packages/shared/tsconfig.json` , and add `transpilePackages: ["@scheduleads-app/shared"]` to the frontend config. Generate the first migration. **Done when:** `npm run build --workspace=backend` succeeds, `npm run build --workspace=frontend` succeeds, and a generated migration file under `packages/shared/drizzle/` contains the `plan` column with its `'agency'` default.
- ☒ **2. Better Auth on the Hono backend.** `backend/src/database.ts` holds the pool; the schema comes from `@scheduleads-app/shared/db` . Write `backend/src/lib/auth.ts` : `drizzleAdapter` with `provider: "pg"` , `emailOTP` , `organization` and `admin` . Mount the handler at `/api/auth/*` . Configure, in this file and nowhere else:
 - `plan` as an organization additional field with `input: false` , so it can never be written from a request.

- The owner role without `organization:delete`, through the plugin's access control. Only the platform `admin` role deletes an organization.
- `databaseHooks.session.create.before`, stamping `activeOrganizationId` from the user's membership when they have exactly one. With two or more and no prior choice it stays unset; picking one would be guessing which tenant the user meant.
- A `sendLoginCode` seam. Outside production it prints the code to the server console. In production it throws, so reaching production before item 6 fails loudly instead of pretending the mail was sent. The code is never in a response body in either case.
- `trustedOrigins` from `APP_ORIGIN`. Cookies stay `SameSite=Lax` in development, where frontend and API differ only by port on `localhost`, and become `SameSite=None; Secure` with a cross-subdomain domain from `COOKIE_DOMAIN` in production. Add credentialed CORS for `APP_ORIGIN` on `/api/auth/*` and the authenticated routes. Leave `WIDGET_ORIGINS` for the public widget reads that arrive in item 2, and never allow credentials on those. **Done when:** the backend starts, requesting a sign-in code for a new email prints the code to the server console, the response body contains no code, and a `verification` row exists for that address.
- ☒ **3. Active organization and the role guard.** Write `backend/src/lib/active-organization.ts`, ported from the first repo's `frontend/lib/active-organization.ts` with the Next adapters swapped for Hono ones: session first, then the active member, then a membership lookup as the fallback, returning nothing when the user has two memberships and no active choice. Add the Hono middleware that puts `{ userId, organizationId, role }` on the context, and `requireOrgRole("owner")` for the owner-only actions items 3 and 12 will need. The organization id is derived from the session and never read from a body, query string or path parameter; write that rule as a comment where the middleware sets it, because this is where it starts. **Done when:** an unauthenticated request to a guarded route returns 401, a signed-in user with one membership resolves to that organization, and a user with two memberships and no active organization is refused rather than assigned one.
- ☒ **4. The package gate.** `packages/shared/src/config/plan-limits.ts` is a pure config module with no database import, holding the one rung `agency` and the modules it unlocks. An unrecognized or null plan resolves to a locked set with no modules, not to `agency : plan` is server-set, so an unknown value is a misconfiguration and the gate fails closed. `backend/src/lib/plan-gate.ts` reads the active organization's `plan` and refuses with 403 and a stable error shape. Add `GET /me` behind auth, the org middleware and the gate, returning the user, the organization, the role, the plan and the resolved limits, and nothing about any other organization. **Done when:** `GET /me` as a signed-in owner returns that organization and `plan: "agency"`, and setting the same organization's `plan` to an unrecognized value directly in the database makes the identical request return 403 with the stable error shape.
- ☒ **5. Sign-in and create-organization on the frontend.** Add `better-auth` to the `frontend` workspace at the version already resolved for the backend. `frontend/lib/auth-client.ts` builds the client against `NEXT_PUBLIC_API_URL` with `organizationClient()`, `emailOTPClient()` and `adminClient()`, and sends credentials on every call. Build `/sign-in` as the two-step email then

code flow, and `/create-organization` for the first organization. The signed-in shell reads `/me` and renders four states: the organization and its plan, a not-signed-in redirect, the refusal state when the gate returns 403, and the pick-an-organization state when the session resolves to no active organization. Email and code validation come from Zod schemas in `packages/shared` so the two sides cannot drift. **Done when:** `npm run build --workspace=frontend` succeeds and, with both dev servers running, a new email signs in with the console-printed code, arrives at `/create-organization`, creates an organization, and lands on a signed-in page showing that organization's name and the `agency` plan.

- ☒ **6. The second organization, and the env and command record.** Create a second organization as a second user, entirely through the app. Confirm each session sees only its own. Update `.env.example`: `BETTER_AUTH_URL` becomes the API origin rather than `http://localhost:3000`, and `NEXT_PUBLIC_API_URL`, `APP_ORIGIN`, `COOKIE_DOMAIN` and `WIDGET_ORIGINS` join it. Drop the ImageKit and S3 blocks if nothing in the plan claims them, or leave them and say why. Record `db:generate` and `db:migrate` under Commands in `AGENTS.md`. Promote Frank's own user row to the `admin` role through `drizzle-kit studio` and note in the spec archive that no request path can set that field. **Done when:** two organizations exist, both created only through the app, `GET /me` for each session returns only its own organization, and both workspace builds pass.

Files / areas

- `packages/shared/package.json`, `tsconfig.json`, `drizzle.config.ts` - new exports, build script and Drizzle config.
- `packages/shared/src/db/schema/auth-schema/*`, `src/db/index.ts` - the Better Auth tables and their barrel.
- `packages/shared/src/config/plan-limits.ts` - the rungs, no database import.
- `packages/shared/src/validation/auth.ts` - the email and code Zod schemas.
- `packages/shared/drizzle/` - generated migrations, committed.
- `backend/src/database.ts` - the pool.
- `backend/src/lib/auth.ts` - the Better Auth instance and every plugin choice.
- `backend/src/lib/active-organization.ts`, `backend/src/lib/plan-gate.ts` - the two guards.
- `backend/src/index.ts` - CORS, the auth handler mount, `GET /me`, `/health`.
- `backend/package.json` - the `prebuild` hook.
- `frontend/lib/auth-client.ts`, `frontend/app/sign-in/`, `frontend/app/create-organization/`, the signed-in layout.
- `frontend/app/globals.css` - the tokens ported from `prototypes/theme.css`.
- `frontend/next.config.ts` - `transpilePackages`.
- `frontend/package.json` - `better-auth`.
- `.env.example`, `AGENTS.md` - step 6.

Data / contracts

- All ids text, Better Auth style. All timestamps `timestamptz`.
- `organization.plan`: text, not null, default `'agency'`. Server-set only, `input: false`. The only writer before Phase 9 is Frank, by hand.
- `user.role`: the `admin` plugin's field. Server-set only. No request path may write it in this feature or any later one before item 23.
- `session.activeOrganizationId`: set by the session-create hook when the user has exactly one membership, otherwise left unset.
- Plan resolution: a recognized rung returns its limits; anything else, null included, returns the locked set. The lookup lives in one function in `plan-limits.ts` and nothing else reads the raw string.
- `GET /me` returns `{ user: { id, email, name }, organization: { id, name, slug, plan }, role, limits }`. No field from any other organization appears in any response in this feature.
- Refusals: 401 unauthenticated, 403 for a role or plan refusal, both with the same JSON error shape so later items do not each invent one. Fix the shape in step 4 and reuse it.
- Cookies: `SameSite=Lax` in development, `SameSite=None; Secure` with the `COOKIE_DOMAIN` value in production. The frontend and the API must sit under one registrable domain in production or the session cookie is dropped.

Testing

No unit test runner is configured and `verification.logicTests` is `when-configured`, so this feature adds no automated tests and claims none. Each step's `Done when` is observed by hand against the running app and the two workspace builds. `/tests` can add a runner later; the plan-limits resolver and the active-organization fallback are the two pieces worth covering first when it does.

No browser test command exists, so no browser coverage is claimed.

Notes for the AI

- The first repo put Better Auth inside Next; this repo puts it on Hono, so items 1 to 3 are ports of the reasoning, not of the files. Carry over the comments that explain *why* - the two-membership refusal, the wall around the secret, the never-from-the-request rule - and rewrite the adapters around them.
- Read these before writing, they are the proven shapes: `../scheduleads/frontend/lib/auth.ts`, `../scheduleads/frontend/lib/active-organization.ts`, `../scheduleads/packages/shared/package.json`, `../codestash/lib/config/plan-limits.ts`, `../codestash/lib/actions/get-org-plan-limits.ts`.
- `better-auth`, `drizzle-orm`, `postgres`, `drizzle-kit` and `resend` are already installed in `backend`. Step 1 declares `drizzle-orm`, `zod`, `drizzle-kit` and `typescript` on

`packages/shared` and step 5 adds `better-auth` to `frontend`; every one of those is already in the lockfile, so nothing new enters the tree. Ask before anything else.

- Next 16 has breaking changes against older training data. Read the guide in `node_modules/next/dist/docs/` before writing framework code.
- Do not assert Better Auth endpoint paths from memory. Read them off the installed version's types when wiring step 5.
- Development runs against a local PostgreSQL 18, `scheduleads_dev`, since 2026-09-23. Railway is reached through its tunnel only on purpose; when port 5433 listens but every query resets, restart the tunnel.
- The gate on `GET /me` is deliberate. An organization on an unrecognized rung is misconfigured and has no working product, so the dashboard refuses rather than rendering empty. `/me` mounts `requireKnownPlan` only; item 2's first real module route mounts `requireKnownPlan` and then `requireModule`. The two layers were one until F-07 showed `/me` enforcing a module check while this note described a recognition check.

Open questions

- **Render or Railway for the backend.** The overview says Railway; you said Render. It changes nothing in steps 1 to 6, which all run on localhost, but the production cookie needs both real hostnames under one registrable domain before the first deploy. Answer it at `/release`, not here.
- **Whether `.env.example` keeps its ImageKit and S3 blocks.** Scaffolder defaults that nothing in the plan currently claims. Step 6 either removes them or records why they stay; your call at that step's review.

Implementation walkthrough

What was actually built, by area. The steps above are the plan; the build log records how each step went against it. This is the shape of the code the item left behind, and the decisions in it that are not visible from the code alone.

packages/shared

- The package became real: it compiles to `dist/` and exposes three subpath exports, `./db`, `./config` and `./validation`. Both apps build it first through their `predev` and `prebuild` hooks, so neither consumes it as TypeScript. The first repo exported source, which worked under `tsx` and would have broken the first time a built server imported a `.ts` file.
- `src/db/schema.ts` is one file rather than a directory behind a barrel, because the two workspaces disagree about import extensions. It holds the seven Better Auth tables for the `emailOTP`, `organization` and `admin` plugins, `timestamptz` throughout, `organization.plan` defaulting to

`agency` , and a unique index on `member(organizationId, userId)` so the two-business refusal is backed by the database.

- Migration `0000` adopts the first repo's tables with `ALTER` s, because the database arrived holding a real tenant. Since F-06 it first creates all seven tables `IF NOT EXISTS` , so the same ledger builds an empty database, and since F-01 it has the snapshot that lets `db:generate` diff correctly. This repo keeps its own ledger table, leaving the first repo's history untouched.
- `src/config/plan-limits.ts` is pure config with one rung. Anything unrecognised resolves to the locked set, and `isKnownRung` uses `Object.hasOwn` because `in` accepted `constructor` and `toString` .
- `src/validation/auth.ts` holds the email, code and business-name rules and the slug derivation, imported by both the form and the API.
- `scripts/seed-dev.ts` (`db:seed`) creates one platform admin and one ordinary owner, each with a business. It exists because closed signup leaves a fresh database with no way in, and it refuses any database that is not on this machine or not named `*_dev` , because the Railway tunnel is also localhost.

backend

- `src/lib/auth.ts` mounts Better Auth on Hono. Login codes are stored hashed. Signup is disabled and only the platform admin may create a business, so every account is one the agency created. The owner role cannot delete an organization. A session-create hook stamps the active business when the user has exactly one, which is the org fix carried from the first repo. `APP_ORIGIN` and `BETTER_AUTH_URL` default to localhost in development and refuse to boot without a value in production. Cookies are `Lax` locally and `None; Secure` across subdomains in production.
- `src/lib/send-login-code.ts` prints the code outside production and throws inside it, until item 6 brings a real mail transport.
- `src/lib/active-organization.ts` is where tenant scope starts: the business comes from the session and nothing else. The active member is looked up on the session's user and its active organization together, a single membership is the fallback, and two memberships with no choice are refused rather than guessed. It also owns the one refusal shape and its five codes, and the owner-only role guard.
- `src/lib/plan-gate.ts` is two layers: `requireKnownPlan` refuses a rung that does not exist, and `requireModule` refuses one that lacks the module. The first reads the plan, name and slug in one query.
- `src/index.ts` keeps two CORS policies apart, so the public widget origins can never carry credentials. It mounts `/api/auth` , and `GET /me` answers in three queries with nothing about any other business.

frontend

- `app/globals.css` carries the tokens ported from `prototypes/theme.css` : light by default, dark only by explicit choice.
- `lib/auth-client.ts` is Better Auth's typed client against the API, with credentials on every call. `lib/api.ts` turns `/me` into five states and branches on refusal codes by name.
- `app/sign-in` is email, then code. An address with no account is told a code is on its way and none is sent, deliberately, so the form cannot test whether an address is a customer. `app/create-organization` works only for the platform admin. `app/page.tsx` renders the business, the sign-in redirect, the pick-a-business state and the refusal.

Decisions worth knowing later

- `user.role` and `organization.plan` are both unwritable from a request and fail in opposite ways: a role in a body is refused with a 400, a plan is silently replaced by its default. A 200 on create is not evidence a value was taken.
- The dashboard's front door checks only that the plan exists; each module checks itself. Refusals never assert a cause they have not established.
- Development moved to a local PostgreSQL 18 on 2026-09-23. Only migrations reach Railway.

Carried forward, not done here

- Five small findings from the final review stay open in the live ledger: F-12 (P2) and F-13 to F-16 (P3). None blocks; they go to the items that touch those files.
- Build-plan item 3b, provisioning a client into their business, was found during this item's review and is a prerequisite for Phase 5.
- No test runner exists; the plan resolver and the active-organization fallback are the first two things worth covering.
- The overview's plan fingerprint is behind the plans, including item 3b and the resource-model edits. `/overview` refreshes it; this completion did not rewrite the fingerprint, since that would only hide the drift.

Findings

Resolved during this item and archived with it. IDs carry the item number; the bare IDs inside each entry are the ones used while the work was live.

1/F-01 [P1] closed - Migration 0000 has no snapshot, so the next `db:generate` recreates all seven tables

File: `packages/shared/drizzle/meta/_journal.json:1` **Found:** 2026-09-22 by `/audit` (scope: current; lens: quality) **Why it matters:** The journal records entry `0000_adopt_repo_one_tables`, but `meta/` holds only `_journal.json`. There is no `0000_snapshot.json`. drizzle-kit builds its previous state only from

the non-underscore files in `meta/` (`prepareOutFolder` , `bin.cjs:8135`) and falls back to the empty schema when that list is empty (`bin.cjs:19864`). So the next generate diffs the whole schema against nothing.

Confirmed by running it, not inferred: `drizzle-kit generate` against a scratchpad copy of the folder produced `0001_*.sql` containing seven bare `CREATE TABLE` statements with no `IF NOT EXISTS` , and no trace of any real delta. Applied to the live database, which migration 0000's own comment says already holds all seven tables, that fails on the first statement.

`AGENTS.md:378` documents `db:generate` as the way this project evolves its schema, and build-plan item 2 opens by adding `booking_link` , `availability_rule` , `contact` and `lead` . That is when this fires, and it fires quietly: the generated file looks like an ordinary migration.

The hand-written 0000 is the right call for adopting the first repo's database; the gap is only that the ledger was never given the snapshot that makes the next diff correct.

Suggested fix: Commit the missing `packages/shared/drizzle/meta/0000_snapshot.json` describing the schema as of 0000. Generate it once in a throwaway folder from the current `schema.ts` , keep only the snapshot, rename it to `0000_snapshot.json` , and leave the committed `0000_adapt_repo_one_tables.sql` untouched. Then confirm a following `db:generate` reports no changes. **Resolution:** Fixed 2026-09-22 exactly as suggested. Generated a snapshot of the current schema into a scratchpad folder with an empty `meta/` , which produced a first-in-chain snapshot (`prevId` all zeros, version 7, all seven tables, `organization.plan` with its `'agency'` default, and the `member_organization_user_unique` index). Copied it in as `packages/shared/drizzle/meta/0000_snapshot.json` ; `0000_adapt_repo_one_tables.sql` untouched. Verified by running the real `npm run db:generate --workspace=@scheduleads-app/shared` , which now reports "No schema changes, nothing to migrate" where the same command previously emitted seven bare `CREATE TABLE` statements. No stray migration file was produced.

Closed 2026-09-23 by the second independent review, which tested the claim rather than accepting it. `packages/shared/drizzle/meta/0000_snapshot.json` is present, 15,341 bytes, `prevId` all zeros, version 7. Ran the real `npm run db:generate --workspace=@scheduleads-app/shared` : exit 0, output "No schema changes, nothing to migrate", and drizzle-kit's own summary confirms the snapshot it read describes the current schema (`organization 7 columns` , `member 5 columns 1 indexes`). Recorded the SHA-256 of all three files under `drizzle/` before and after the run; none changed and no new file appeared, so the generate really is a no-op rather than a quiet write. The repair holds. A separate and distinct gap in the same ledger is recorded as F-06: a correct snapshot still does not let the ledger create the schema from nothing.

1/F-02 [P2] closed - The one shared refusal shape cannot be reused, so two of four refusal sites hand-roll it

File: `backend/src/lib/active-organization.ts:41` **Found:** 2026-09-22 by /audit (scope: current; lens: quality) **Why it matters:** The spec's Data / contracts section says refusals use "the same JSON error shape so later items do not each invent one. Fix the shape in step 4 and reuse it." The type is exported but

`refuse` is not, so no other module can reach it. Both remaining refusal sites therefore build the object by hand: `backend/src/lib/plan-gate.ts:48` returns `code: "plan_required"`, which is not a member of `Refusal["error"]["code"]`, and `backend/src/index.ts:73` re-types a `no_active_organization` literal that already exists three lines of code away.

The result is that the contract is documented and not enforced. Every route from item 2 to item 26 adds refusals, and a reader has three examples to copy, only one of them typed. `frontend/lib/api.ts:79` already branches on these code strings with nothing shared to check them against, so a later rename on the server goes undetected on the client.

Suggested fix: Export `refuse`, add `plan_required` to the `Refusal["error"]["code"]` union, and call `refuse` from `plan-gate.ts` and `index.ts` instead of the two literals. Exporting the union from `packages/shared` would also let `frontend/lib/api.ts` branch on a checked type, but that is a larger move and item 2's RPC seam may supply it instead. **Resolution:** Fixed 2026-09-22. `refuse` is now exported from `active-organization.ts`, and the code union is extracted as a named `RefusalCode` type with `plan_required` added as a fourth member, each code carrying a one-line comment saying when it applies. `plan-gate.ts` and the `/me` missing-organization branch in `index.ts` both call the helper instead of hand-rolling the object, so all four refusal sites now go through one function and every code is inside the union. Proved at runtime rather than by the build alone: unauthenticated `/me` returns 401 `unauthenticated`, and flipping a live organization to an unrecognised plan returns 403 `plan_required` with the shared shape, then 200 again once restored. The shared-package move stays out of scope, as the finding suggests.

Closed 2026-09-23 by the second independent review. `refuse` is exported at `active-organization.ts:58`, `RefusalCode` is a named four-member union at lines 34 to 42, and a search for a bare `error:` object literal across `backend/src` returns only the `Refusal` type declaration and the helper's own body, so no call site builds the object by hand any more. All four refusal sites go through the helper: `index.ts:74`, `active-organization.ts:144`, `active-organization.ts:192` and `plan-gate.ts:50`. The backend build passes and unauthenticated `GET /me` against the running API answers 401 with the shared shape. One caveat recorded rather than reopened: the shape covers this API's own routes only. Better Auth owns `/api/auth/*` and answers with its own message and code shape, confirmed live against `/api/auth/organization/list`, so the description at `frontend/lib/api.ts:40` of a shape the API "reuses everywhere" overstates its reach. Harmless today, because `/me` is the only route that helper parses.

1/F-03 [P2] closed - `GET /me` resolves the session three times and reads the same organization row twice

File: `backend/src/lib/active-organization.ts:130` **Found:** 2026-09-22 by `/audit` (scope: current; lens: performance) **Why it matters:** `requireOrganization` resolves the session at line 125, then calls `getActiveOrganization`, which resolves it again at line 63, then calls `auth.api.getActiveMember`, whose `orgSessionMiddleware` runs `sessionMiddleware` and resolves it a third time (`better-auth/dist/plugins/organization/call.mjs:11`). `session.cookieCache` is not configured in `auth.ts`, so each of the three is a database read rather than a cookie check.

Then `plan-gate.ts:38` selects `plan` from the organization row and `index.ts:62` selects `name` and `slug` from that same row in a second query. The comment at `index.ts:58` notices the duplication and leaves it.

This is not a hypothesis about latency, which was not measured; it is the call count, read off the code. It matters because `/me` is the dashboard's bootstrap on every load and because this middleware stack is the one every route from item 2 onward mounts, so the duplication is inherited rather than confined here. Local development runs Postgres over an SSH tunnel, where per-query cost is exactly what is felt first.

Suggested fix: Have `requireOrganization` pass the session it already resolved into `getActiveOrganization` rather than letting it re-derive one, and resolve the active member from the session's `activeOrganizationId` with the membership query already in that file instead of going through `auth.api.getActiveMember`. Separately, let `requireModule` select `plan`, `name` and `slug` in its one query and put the row on the context for the handler to read. **Resolution:** Re-confirmed 2026-09-23 by the second independent review and left `open`, unchanged. All of it is still there: the three session resolutions at `active-organization.ts:142`, `active-organization.ts:80` and inside `auth.api.getActiveMember`, no `session.cookieCache` in `auth.ts`, and the organization row read twice at `plan-gate.ts:39` and `index.ts:62`. The repair commit deliberately left it, recording that the cookie cache keeps a revoked session alive for the cache window and is therefore a decision rather than a cleanup. That reasoning is sound, and the suggested fix above needs no cookie cache to work. P2, so it does not block the merge.

Fixed 2026-09-23 as the finding suggested, and without the cookie cache, whose revocation window stays a separate decision. `/me` goes from six database queries to three. Counted off the code and the installed better-auth 1.7.5, where one session resolution is one query: `findSession` in `internal-adapter.mjs` is a single `findOne` joining `user`, with no secondary storage configured to change that.

- Before: three session resolutions (`requireOrganization`, `getActiveOrganization`, and `getActiveMember`'s own middleware), its member lookup, and the organization row twice. Six.
- After: one session resolution, one member lookup, one organization read. Three.

`getActiveOrganization` now takes the session its caller already resolved, not the headers. `auth.api.getActiveMember` is replaced by the lookup it performed, read off `crud-members.mjs:396-402`: the member row filtered on the session's user **and** its active organization, with a missing row falling through to the membership lookup just as its `MEMBER_NOT_FOUND` did. Both filters are kept on purpose and the reason is written where they sit. `getActiveOrganization` had one caller, so the signature change reaches nothing else.

`requireKnownPlan` selects `plan`, `name` and `slug` in its one query and hands the name and slug on as `organizationDetails`. The raw plan string is not handed on, so `plan-gate.ts` stays the only place it is read. The `/me` handler makes no query of its own now, and its missing-row branch is gone because the gate already refuses that case, since F-07, with `no_active_organization`. `index.ts` sheds its `db`, `eq`, `organization` and `refuse` imports.

Backend build passes under strict mode, which also confirms

`session.session.activeOrganizationId` exists on the inferred session type rather than being

assumed. Not proved against the running API: the `/me` body should be byte-for-byte what it was, and the independent review is the place to confirm that with a signed-in session, alongside F-05 and F-07.

Seen live on 2026-09-23 by Frank, walking the running app himself against the local development database (`scheduleads_dev` , PostgreSQL 18.6, seeded by `db:seed`), before any fresh review. Recorded as evidence for that review, not as a closure. `/me` returned every field for both seeded owners: the business name and slug, the plan, the owner role, and both modules, which is the body this finding's repair was required to leave unchanged.

Closed 2026-09-23 by the third independent review (fresh subagent). Counted off the code at `9c79839` : `requireOrganization` resolves the session once (`active-organization.ts:171`) and hands it to `getActiveOrganization` , which makes one member query filtered on user and organization (`:107-116`) and falls back to one membership query only when that misses (`:127-131`). No `auth.api.getActiveMember` call remains anywhere in `backend/src` . `requireKnownPlan` reads `plan` , `name` and `slug` in one select (`plan-gate.ts:293-301`) and the `/me` handler makes no query of its own (`index.ts:57-78`). The `/me` field mapping is the one the spec's contract names. Not re-observed with a signed-in session here, because login codes print only to the user's API console; Frank's walkthrough above is the runtime evidence.

1/F-04 [P3] closed - The spec ticks step 1 including `transpilePackages` , which was deliberately dropped

File: blueprint/context/current-feature.md:96 **Found:** 2026-09-22 by /audit (scope: current; lens: quality)

Why it matters: Step 1 instructs adding `transpilePackages: ["@scheduleads-app/shared"]` to the frontend config, and Files / areas lists `frontend/next.config.ts` for it. The box is ticked and `frontend/next.config.ts` is untouched scaffold with no `transpilePackages` ; the file does not appear in the feature's diff at all.

The decision itself is sound and is recorded properly on the published build log as `dropped` , with its reason, at `project-log.html:1455` . Nothing is hidden from the person who reads the page. The gap is only in the spec, whose own preamble records two other deviations "rather than quietly corrected" and is silent on this third one. The spec is what `/complete` archives and what the review hash pins, so it is the copy that outlives the page's current state.

Suggested fix: Add one line to the spec's Status preamble naming the dropped piece and pointing at the build log entry, in the same form as the two deviations already listed there. **Resolution:** Fixed 2026-09-22. A third bullet added to the spec's Status preamble naming `transpilePackages` as deliberately dropped, why it was dropped (once `packages/shared` compiled to JavaScript the setting did nothing), that `frontend/next.config.ts` is therefore untouched scaffold, and that the build log carried it at the time while the preamble did not. Note this changes the spec bytes, so the review receipt's spec hash no longer matches and a fresh review is required regardless.

Closed 2026-09-23 by the second independent review. The third bullet is present at `current-feature.md:22-27` and says all of that. Confirmed the state it describes rather than the claim: `frontend/next.config.ts` is seven lines of untouched scaffold with no `transpilePackages` , and both

workspace builds pass without it, so the dropped setting genuinely does nothing. The specific gap this finding named is gone. Two other spec-to-reality gaps survived the amendment and are recorded separately as F-10 rather than keeping this entry open.

1/F-05 [P1] closed - Any email address can create an account and unlimited tenants on the `agency` rung

File: `backend/src/lib/auth.ts:110` **Found:** 2026-09-23 by /audit (scope: current; lens: security) **Why it matters:** This feature's stated Goal is that it "decides where the trusted actor comes from" and that "the boundary it draws is the one the product keeps". The boundary it draws admits anyone.

Three permissive defaults are left untouched, each read off the installed better-auth 1.7.5 rather than assumed:

- `emailOTP` is configured without `disableSignUp`. `routes.mjs:102` computes `shouldSendOTP` as `type === "sign-in" && !opts.disableSignUp`, and `routes.mjs:412` refuses account creation only when `disableSignUp` is set. So a `sign-in` code is issued for an address that has no account, and verifying it creates the user.
- The `organization` plugin is configured without `allowUserToCreateOrganization`. `crud-org.mjs:56` resolves an undefined option to `true`, so every authenticated user may create an organization.
- `organizationLimit` is also unset. `crud-org.mjs:61` evaluates the whole cap expression to `false` when the option is absent, so there is no per-user ceiling either.

New organizations take `plan: "agency"` from the `defaultValue` at `auth.ts:135`, and `plan-limits.ts:31` gives `agency` both `booking` and `crm`. The reachable path is therefore: a stranger requests a code for any address they control, verifies it, gets a user row, then posts to `/api/auth/organization/create` as many times as they like, each one a tenant on the paid rung.

That contradicts a written product decision, not a matter of taste. `project-overview.md:254` says "Agency-provisioned first ... Frank creates the organization and bills as the agency"; `project-overview.md:32` puts the self-serve customer in Phase 9; `build-plan.md:189` is item 25, "Self-serve onboarding and billing - signup, pick a package". `AGENTS.md:16` says the same thing with a different phase number. Nothing anywhere asks for open signup now.

The timing is the dangerous part. It is not exploitable in production today, because `send-login-code.ts:30` throws in production, so no code is ever issued there and nobody can sign in at all. It becomes exploitable the moment build-plan item 6 swaps Resend into that seam, with no other change. Item 6's charter is transactional email; nobody working it will be looking at signup policy, and the spec itself describes that step as the send seam being "swapped". A guard that has to be remembered during an unrelated item is a guard that will not be there.

Suggested fix: Two options on the `betterAuth` call in `backend/src/lib/auth.ts`, one line each. On `emailOTP`, set `disableSignUp: true` if only existing users should ever sign in. On `organization`, set `allowUserToCreateOrganization` to a function that returns true only for the platform admin until item 25 relaxes it, and set `organizationLimit` to a real number. Whichever is chosen, say so at the

declaration, because the next reader will assume the permissive default was deliberate. If Frank decides the risk window is acceptable and the guard belongs to item 6 or item 25, that is his call to record as `accepted` with the reason; it should not be closed silently.

Resolution: Fixed 2026-09-22 in `3030ef0`, partially, and the rest deliberately deferred rather than silently dropped. The finding named three permissive defaults; they did not all deserve the same answer.

- `allowUserToCreateOrganization` is now a function returning true only when `user.role === "admin"`, at `backend/src/lib/auth.ts:136`, with the reason written at the declaration as the finding asked. This is the half that was actually a hole: it is what stood between a stranger and an unlimited supply of `agency`-rung tenants. The commit records a live proof, signed in as a non-admin through the app and posting to `organization/create`: 403 `YOU_ARE_NOT_ALLOWED_TO_CREATE_A_NEW_ORGANIZATION`, organization count unchanged afterwards.
- `disableSignUp` is now set, 2026-09-23, a day after the first half. It was briefly left open on the argument that a client being onboarded needs to sign in and has no user row yet. That is true in general and was wrong here: it assumed someone was mid-onboarding, and nobody is. Every user who needs to sign in today already exists, so the door had no legitimate user at all, while it did have a cost - this API could be made to send mail to any address a stranger named, which is what item 6 turns real by putting Resend behind the send seam. Existing users are unaffected, verified against `routes.mjs:103`, which short-circuits only when `findUserByEmail` comes back empty. The deliberate consequence is that a new client now cannot get in at all without the agency, which is what build-plan item 3b exists to provide.
- `organizationLimit` is still unset and is now moot. It caps organizations per user, and the only user who may create one is the platform admin, so the cap would only ever restrain Frank. Left off deliberately; if item 25 ever reopens creation to ordinary users it has to come back in the same edit.

Verified beyond the commit's own claim, against the installed better-auth 1.7.5 source rather than its docs. `crud-org.mjs:57` resolves the option and refuses when it returns false. Two bypasses were checked and neither is reachable over HTTP: a signed-in non-admin cannot impersonate by putting `userId` in the body, because `user` is taken from the session and the permission check runs against them (`crud-org.mjs:48-55`); and the `isSystemAction` escape at `crud-org.mjs:57-58`, which does skip the check, sits behind `if (!session && (ctx.request || ctx.headers)) throw UNAUTHORIZED` at `crud-org.mjs:47`, and both of those are always present on an HTTP call. That escape exists for direct server-side `auth.api` calls with no headers, which nothing in this codebase makes today. Worth carrying into item 3b: it is the sanctioned provisioning path, and it is also the one place a later server-side caller could create an organization without the admin check, so that item should say so where it uses it.

Not re-proved at runtime in this pass: the dev API was not running and bringing it up needs the Railway SSH tunnel. The live 403 recorded above is the builder's, from the fix commit. An independent review pass should re-run it before moving this to `closed`.

Seen live on 2026-09-23 by Frank, walking the running app himself against the local development database (`scheduleads_dev`, PostgreSQL 18.6, seeded by `db:seed`), before any fresh review. Recorded as evidence for that review, not as a closure. Signed in as `owner@example.com`, an ordinary owner, and submitted `/create-organization`: refused with "You are not allowed to create a new organization". The

business count was read from the database afterwards and was still two, so the refusal left nothing behind. The other half held too: `nobody-sep23@example.com`, an address with no account, was told a code was on its way, none was generated, and any code returned "Invalid OTP".

Closed 2026-09-23 by the third independent review (fresh subagent), both halves re-examined against the code and the installed better-auth 1.7.5.

- Signup: `disableSignUp: true` at `backend/src/lib/auth.ts:232`. In `email-otp/routes.mjs:102-106`, `shouldSendOTP` is false whenever it is set, so an address with no user row gets `{ success: true }` and nothing is sent. Re-proved against the running API: `send-verification-otp` for an unknown address answered 200 `{"success":true}`, and `sign-in/email-otp` for the same address answered 400 `INVALID_OTP`. `/sign-up/email` is refused too, because `emailAndPassword` is not enabled (`api/routes/sign-up.mjs:144`).
- Organization creation: `allowUserToCreateOrganization` returns true only for `role === "admin"` (`auth.ts:174-175`), and `crud-org.mjs:56-58` refuses when it returns false; the `isSystemAction` escape needs no session and no request, which an HTTP call never satisfies (`crud-org.mjs:47`). Unauthenticated `organization/create` against the running API answered 401. The non-admin 403 itself was not re-run here, since signing in needs a code from the user's console; Frank's walkthrough above observed it with the count unchanged.

The original path, a stranger minting a user and then unlimited `agency` tenants, is gone. Paths the plugins still expose that the spec reserves for later items are recorded separately as F-16; none of them admits a stranger.

1/F-06 [P2] closed - The migration ledger can only alter tables, so `db:migrate` fails on any fresh database

File: `packages/shared/drizzle/0000_adapt_repo_one_tables.sql:15` **Found:** 2026-09-23 by /audit (scope: current; lens: quality) **Why it matters:** Migration 0000 is entirely `ALTER TABLE` plus one `CREATE UNIQUE INDEX`. There is no `CREATE TABLE` anywhere under `packages/shared/drizzle/`. Postgres applies `ADD COLUMN IF NOT EXISTS` to the column, never to the table, so line 15's `ALTER TABLE "user" ADD COLUMN IF NOT EXISTS "role" text` errors with `relation "user" does not exist` against an empty database, and the migration aborts on its first statement.

Adopting the first repo's tables instead of recreating them was the right call for the one database that exists, and F-01's snapshot now makes the ledger diff correctly going forward. This is the other half of the same gap: the ledger can evolve that database but can never produce it. There is no sanctioned substitute either, because `coding-standards.md` forbids `drizzle-kit push` against anything but a local scratch database.

`AGENTS.md:379` records `db:migrate` as how this project applies schema, and this fires the first time anyone stands up a staging database, a CI database, or a replacement Railway instance. Deployment is the next thing after this feature: the spec's own Open questions leave Render versus Railway to `/release`.

Not reproduced against a real empty database, because this review may not create one or modify any row. The conclusion is read off the SQL and Postgres's documented behaviour for `ADD COLUMN IF NOT EXISTS` , which is unambiguous.

Suggested fix: Nothing in migration 0000 should change; it is applied. Add `0001` as the bootstrap instead, generated from the current schema into a throwaway folder so it is the seven `CREATE TABLE IF NOT EXISTS` statements plus their constraints, then hand-checked so it is a no-op against the live database and a full create against an empty one. Confirm both directions before committing, and say at the top of the file that it exists because 0000 adopts rather than creates.

Resolution: Fixed 2026-09-23, and **not** as suggested, because the suggested fix does not work. Drizzle applies pending migrations in journal order inside one transaction (`drizzle-orm/pg-core/dialect.js` , `migrate()` , which `drizzle-kit migrate` reaches through `postgres-js/migrator.js`). On an empty database `0000` runs first and aborts on its first `ALTER TABLE` , so a bootstrap numbered `0001` would never be reached. The finding's diagnosis was right and its repair would have failed exactly the way the diagnosis describes.

The `CREATE TABLE IF NOT EXISTS` block went into the top of `0000` instead, which means editing an applied migration, contrary to the finding's "nothing in migration 0000 should change". Safe here for three reasons, each read off the installed drizzle-orm 0.45.2 rather than assumed, and each written into the file's own header:

- The migrator skips any migration whose journal `when` is not later than the newest `created_at` in the ledger, so the live database never runs `0000` again.
- It stores each file's hash and never compares it afterwards, so the edit trips nothing.
- Every added statement is `IF NOT EXISTS` , so even a re-run against the live database would change nothing.

The seven tables were generated from `schema.ts` with `drizzle-kit generate` into a scratch folder, not typed by hand, and a script confirmed every column and constraint of the new block matches that reference. Foreign keys are inline rather than drizzle's separate `ADD CONSTRAINT` , since an inline constraint is skipped along with its `CREATE` when the table exists and `ADD CONSTRAINT` has no `IF NOT EXISTS` . Drizzle's own constraint names are kept, so later generated migrations can refer to them. `db:generate` still reports no changes.

Proved against a real empty database, not read off the SQL. PGLite, which is Postgres 18.3 compiled to WebAssembly, was installed into a scratch folder only, with drizzle-orm pinned to this project's 0.45.2 so the migrator under test is the one `db:migrate` runs. The project's dependencies are unchanged. Each case used a fresh in-memory database, with the real migrations folder, ledger table and schema from `drizzle.config.ts` :

1. Empty database: migrates cleanly, and the result is identical to `schema.ts` applied directly: 7 tables, 60 columns, 57 constraints, 11 indexes, compared from `information_schema` , `pg_constraint` and `pg_indexes` . The ledger holds one row.
2. Run again: a no-op. Ledger still one row, schema unchanged.
3. The **original** `0000` on an empty database fails with `relation "user" does not exist` , as this finding said, and the rolled-back transaction leaves no tables behind. This is the case that shows the test

can fail at all. 4. Tables already present and an empty ledger: `0000` runs cleanly over them and changes nothing, so every statement in it is safe over existing tables.

Two limits, stated rather than hidden. Production's Postgres version is not known here and PGLite is 18.3; nothing in this file is version-sensitive, but that is a claim, not a test. And no test can say whether the **live** database matches `schema.ts`, because it was built by the first repo. A different constraint name or an extra column there would not show up in any check above. Comparing it takes the SSH tunnel, a `drizzle-kit pull` into a scratch folder and a diff, and it belongs with the review run that already needs the tunnel.

Proved again on 2026-09-23 by real use rather than a test. The local development database was created empty on PostgreSQL 18.6 and built with the ordinary `npm run db:migrate`. It then matched `schema.ts` applied directly: 60 columns, 57 constraints, 11 indexes, with one row in the ledger. Railway was confirmed to run `postgres-ssl:18` from its own deployment metadata, so the version caveat above is closed: both the PGLite test and this build ran on the production major version. Whether the live Railway database matches `schema.ts` is still unchecked.

Closed 2026-09-23 by the third independent review (fresh subagent). Re-read `0000_adapt_repo_one_tables.sql` at `9c79839`: all seven `CREATE TABLE IF NOT EXISTS` statements precede the first `ALTER TABLE`, in dependency order (`user`, `organization`, `verification`, then `session`, `account`, `member`, `invitation`), with foreign keys inline so they are skipped with their table; every later statement is `IF NOT EXISTS` or a same-type `ALTER COLUMN`, and the unique index is `IF NOT EXISTS`. The journal still has one entry and neither `schema.ts` nor `0000_snapshot.json` has changed since F-01 closed (`git log` shows both last touched in `ec5167d`). Not re-run against an empty database here, since this review may not create one; the local `scheduleads_dev` build by `db:migrate` recorded above is the runtime evidence. The live Railway comparison stays an open risk, not a reason to keep this open.

1/F-07 [P2] closed - `/me` refuses any rung that lacks `crm` and tells the user their plan is unrecognized

File: `backend/src/index.ts:53` **Found:** 2026-09-23 by `/audit` (scope: current; lens: quality) **Why it matters:** The rule the code says it implements and the rule it implements are different rules.

Stated in three places: `index.ts:46` says "A business on a rung nothing recognises is misconfigured and has no working product"; the spec's Notes for the AI say "An organization on an unrecognized rung is misconfigured ... so the dashboard refuses rather than rendering empty"; and the dashboard copy at `frontend/app/page.tsx:77` tells the user "Your business is on a plan this dashboard does not recognise".

Implemented: `requireModule("crm")`, which refuses whenever the resolved limits do not include `crm`, recognized or not.

Today the two coincide, because `agency` is the only rung and it carries both modules. Build-plan item 23 is "the rungs above `agency` in the plan-limits config, what each unlocks", and `project-overview.md:255` says "each rung unlocks modules through the plan-limits config". The first rung that

unlocks `booking` without `crm`, which is the obvious shape of a cheaper booking-only tier, locks that tenant out of the dashboard entirely and shows them a sentence that is false: their plan is recognized, it just does not include the CRM. `index.ts:53` is also the middleware stack item 2 onward copies, so the conflation propagates rather than staying here.

Suggested fix: Decide which rule is wanted and make one of the two match. Either gate `/me` on the rung being recognized rather than on a module, letting the dashboard render whatever the plan does include, or keep the `crm` gate and rewrite the three comments and the user-facing sentence to say what it actually does. In either case the frontend refusal should render the API's own message, which it already receives at `api.ts:84`, rather than asserting a cause.

Resolution: Fixed 2026-09-23, choosing the first of the finding's two options: the dashboard's front door asks only that the rung be recognised, and each module's own route checks what the rung unlocks. Frank's call. The other option, keeping the `crm` check and rewording the comments, would have locked out the first booking-only tier entirely, and a price ladder is the next thing item 23 builds.

The gate is now two layers in `backend/src/lib/plan-gate.ts`. `requireKnownPlan` reads the rung and refuses one the config does not define. `requireModule` reads what the first put on the context and refuses a rung that lacks the module, costing no query of its own. `/me` mounts only the first. Each throws if the layer before it is missing, the pattern `requireOrgRole` already used.

Three things came with it, each so that a refusal never asserts the wrong cause:

- A new refusal code, `plan_unrecognised`, kept apart from `plan_required`. They mean different things to a customer, and the dashboard's refusal screen was already claiming the first while the code sent the second.
- A missing organization row now answers `no_active_organization` from the gate, where it used to fall through to a plan refusal. The session pointing at a deleted business is not a plan problem.
- `frontend/lib/api.ts` now branches on `plan_unrecognised` by name. Any other 403 falls through to the unexpected-status answer instead of being shown the "plan not recognised" screen, which is what the finding's last sentence asked for.

Found alongside, and fixed in the same edit because it was the same line: `getPlanLimits` tested `plan` in `PLAN_LIMITS`, and `in` walks the prototype. `"constructor"`, `"toString"` and `"__proto__"` all passed it and resolved to functions with no `modules`, so a plan hand-edited to any of them would have crashed the gate with a 500 rather than failing closed with a 403. Now `Object.hasOwn`, behind a new exported `isKnownRung`. Tested against the compiled `dist/`, not the types: `agency` is known with both modules, and `toString`, `constructor`, `__proto__`, `enterprise`, the empty string, null and undefined all resolve to unknown and no modules.

Backend build, frontend build and lint pass. Not proved against the running API in this pass: that needs the SSH tunnel and a signed-in session, and flipping a live organization's plan to prove the 403. The independent review that closes F-05 needs the same setup and should re-run the plan flip here, now expecting `plan_unrecognised` rather than `plan_required`.

Seen live on 2026-09-23 by Frank, walking the running app himself against the local development database (`scheduleads_dev`, PostgreSQL 18.6, seeded by `db:seed`), before any fresh review. Recorded

as evidence for that review, not as a closure. Both seeded owners, on `agency`, got through the front door. With Test Salon's plan set to `bogus` in the database, the same owner got "This account needs attention" carrying the API's `plan_unrecognised` message, "This business is on a plan the product does not recognise." Restored to `agency` afterwards and confirmed from the database.

Closed 2026-09-23 by the third independent review (fresh subagent). `/me` mounts `requireOrganization` and `requireKnownPlan` only (`index.ts:57`); `requireKnownPlan` refuses an unrecognised rung with `plan_unrecognised` (`plan-gate.ts:316-324`) and a missing row with `no_active_organization` (`:306-314`); `requireModule` is a separate layer that reads the context and refuses with `plan_required` (`:335-356`). `isKnownRung` uses `Object.hasOwn` (`plan-limits.ts:152-154`). `frontend/lib/api.ts:85` branches on `plan_unrecognised` by name and sends any other 403 to the unexpected-status state. The plan flip was not repeated here, because it writes to the database and needs a signed-in session; Frank's walkthrough above is the runtime evidence. The repair introduced nothing new that this pass found.

1/F-08 [P2] closed - `APP_ORIGIN` and `BETTER_AUTH_URL` fall back to localhost in production instead of refusing to boot

File: `backend/src/index.ts:27` **Found:** 2026-09-23 by /audit (scope: current; lens: security) **Why it matters:** `backend/src/index.ts:27` and `backend/src/lib/auth.ts:36` both default `APP_ORIGIN` to `http://localhost:3000`, and `auth.ts:74` defaults `BETTER_AUTH_URL` to `http://localhost:3001`. That value becomes the single credentialed CORS origin (`index.ts:29-34`) and Better Auth's only trusted origin (`auth.ts:89`). A production deploy that misses the variable therefore grants `http://localhost:3000` the right to send credentials to the live API, which is narrow but real: anything the victim runs on that port could call the API with their session attached.

The same two files throw for `BETTER_AUTH_SECRET` (`auth.ts:27`) and `DATABASE_URL` (`database.ts:16`), so the pattern here is inconsistent as well as fail-open. `frontend/lib/auth-client.ts:29-42` argues carefully for its own fallback, and that argument is specific to `NEXT_PUBLIC_*` values being inlined at build time. Neither backend variable is inlined; both are read at runtime, where a throw is a failed boot rather than a failed local build.

Recorded honestly: the blast radius is limited, because a production deploy missing `APP_ORIGIN` is also a broken deploy. `trustedOrigins` would reject the real dashboard origin, so the app would not work and the mistake would be noticed. The finding is the direction of the failure, not a live exposure. CORS itself is correctly scoped today: a request carrying an unlisted Origin gets no `Access-Control-Allow-Origin` header back, verified against the running API.

Suggested fix: Throw for both when `NODE_ENV === "production"`, beside the two checks that already do, and keep the localhost default for development only. `/release` owns deployment readiness and is a natural second place to check them, but a check in the code is what makes the boot fail rather than the review.

Resolution: Fixed 2026-09-23 as suggested. Both variables now go through one helper in `backend/src/lib/auth.ts`, `settingWithDevDefault`, which returns the value when set, the localhost

default in development, and throws in production. That matches the two checks the same files already made for `BETTER_AUTH_SECRET` and `DATABASE_URL`, so the pattern is consistent as well as closed. The reasoning, including why the frontend's `NEXT_PUBLIC_API_URL` keeps its fallback while these do not, is written at the helper.

Went one step past the suggestion. `APP_ORIGIN` was read twice, at `index.ts:27` and `auth.ts:36`, each with its own copy of the default. It is now read once, exported from `auth.ts`, and imported by `index.ts` for CORS, so the credentialed CORS origin and Better Auth's trusted origin cannot drift apart. An empty string counts as unset, because `APP_ORIGIN=` left blank in a platform's env editor is a mistake rather than an origin.

Proved by running the compiled module, not inferred from the diff. Imported `backend/dist/lib/auth.js` directly with no `.env` loaded, a dummy secret and a database URL pointing nowhere, under five environments:

- production with neither set: refused, naming `APP_ORIGIN`
- production with only `APP_ORIGIN`: refused, naming `BETTER_AUTH_URL`
- production with `APP_ORIGIN` set to the empty string: refused, naming it
- production with both set: booted, `appOrigin` the configured value
- development with neither set: booted, `appOrigin` `http://localhost:3000`

`/release` remains a natural second place to check these, but the boot now fails without it, which is the direction the finding asked for.

Closed 2026-09-23 by the third independent review (fresh subagent). `settingWithDevDefault` (`auth.ts:54-65`) returns a non-empty value, the localhost default outside production, and throws in production; it feeds both `appOrigin` (`:72`) and `baseUrl` (`:110`). `index.ts` no longer reads `APP_ORIGIN` itself and imports `appOrigin` for its CORS policy (`index.ts:6`, `:27-32`). Against the running development API a preflight from `https://evil.example` got no `Access-Control-Allow-Origin`, a preflight from `http://localhost:3000` got it, and a POST to Better Auth from the foreign origin was refused 403 `INVALID_ORIGIN`. The production-boot refusal was not re-run here; it would mean starting a process.

1/F-09 [P2] closed - `coding-standards.md` now contradicts the shipped code in three places

File: blueprint/context/coding-standards.md:52 **Found:** 2026-09-23 by /audit (scope: current; lens: quality) **Why it matters:** `AGENTS.md` lists this file as "read before changing code". Feature 1 made three of its statements false and none was updated, so the next agent that obeys it will undo a deliberate decision.

- The `packages/shared` paragraph says the package "exposes subpath exports that point at source files, no barrel". They point at `dist/` (`packages/shared/package.json:6-19`), and `AGENTS.md:387` now states the opposite of the standard: "`packages/shared` compiles to `dist/` and both apps build it first". An agent following the standard would add a source-pointing export and break both builds.

- Styling says "Dark mode first, light mode as option". `frontend/app/globals.css` ports the mockups' explicit decision, which is light by default with dark reachable only through `data-theme="dark"` , and says so at length.
- Comments says "No banner/header blocks, section dividers, or step-by-step narration" and "Over-commenting is a common AI tell, so resist it". Every new file in this feature opens with a header block, `globals.css` uses long rules of equals signs as section dividers, and `backend/src/lib/send-login-code.ts` is 17 lines of comment above 18 lines of code.

The comments are not the problem: they carry the reasoning this project deliberately keeps, and the spec's Notes for the AI ask for exactly that. The problem is a standards file that forbids what the project has decided to do, which is the two-levels-contradicting failure the workspace `CLAUDE.md` says must never happen.

Suggested fix: Edit `coding-standards.md` to match what shipped, not the other way round. Correct the `packages/shared` sentence to `dist/` , replace "Dark mode first" with the explicit-toggle decision the mockups actually make, and narrow the Comments section so it forbids comments that restate the code while permitting the file-level why-blocks this codebase is built on.

Resolution: Fixed 2026-09-23, editing the standards to match what shipped rather than the other way round, which is the direction the finding asked for. All three statements were confirmed false before being touched.

- The `packages/shared` paragraph said the subpath exports "point at source files, no barrel". They point at `dist/` (`package.json:8-13`). Rewritten to say so, to keep the extensions reason that made it true once, and to add the warning the old text invited: an export pointing at `src/` breaks both builds.
- "Dark mode first, light mode as option" is the opposite of what shipped. `globals.css:117` defines light on `:root` and `globals.css:215` puts dark behind `[data-theme="dark"]` , with no `prefers-color-scheme` anywhere and no third "system" state. Replaced with the decision the mockups actually make.
- The Comments section forbade "banner/header blocks" and warned that over-commenting is an AI tell, while every file this feature added opens with a why-block. Narrowed rather than deleted: restating the code is still forbidden, region-announcing comments and narration are still forbidden, and file-level blocks carrying a decision or a rejected alternative are now explicitly encouraged. Added the citation habit this feature used throughout, naming the dependency file and line a behaviour was read from.

The comments were never the problem and none were removed. The problem was a file `AGENTS.md` tells the next agent to read before changing code, telling them to undo deliberate decisions, which is the two-levels-contradicting failure the workspace `CLAUDE.md` forbids.

Closed 2026-09-23 by the third independent review (fresh subagent). All three named statements now match the code: the `packages/shared` paragraph says the exports point at `dist/` (`coding-standards.md:46-55` , matching `packages/shared/package.json:6-19`), Styling describes the explicit

`data-theme` toggle, and the Comments section permits why-blocks while still forbidding narration. A fourth contradiction in the same file, which this finding never named, is recorded separately as F-15.

1/F-10 [P3] closed - The spec still lists step 1 pieces that never shipped, and its own deviation count is now wrong

File: blueprint/context/current-feature.md:8 **Found:** 2026-09-23 by /audit (scope: current; lens: quality)

Why it matters: F-04's repair added the missing `transpilePackages` bullet and stopped there. Two gaps of the same kind survive in the same preamble, and the repair introduced a third.

- Line 8 still reads "Two things in this spec turned out to be wrong" above three bullets. The count was not updated when the third was added, and `AGENTS.md` is explicit that a bare count beside a list is how a reader is misled.
- Step 1 (line 98) lists `./db/schema` among the subpath exports and (line 103) says to write the tables "under `src/db/schema/auth-schema/`"; Files / areas (line 206) lists `packages/shared/src/db/schema/auth-schema/*` and `src/db/index.ts`. What shipped is a single `packages/shared/src/db/schema.ts` with no `db/index.ts`, and the third export is `./validation`, which step 1 never names. `schema.ts:19-23` explains the one-file decision well; the preamble does not record it.

Same consequence F-04 named: the spec is what `/complete` archives and what the review hash pins, so it is the copy that outlives the build log's current state.

Suggested fix: Change "Two things" to "Three things", or drop the count and let the bullets speak. Add one bullet naming the one-file schema and the `./validation` export in place of `./db/schema`, pointing at the reason already written in `schema.ts`.

Resolution: Fixed 2026-09-23, both halves.

The count is gone rather than corrected. `AGENTS.md` is explicit that a bare count beside a list misleads, and this one had already gone stale once; a number that has to be maintained in step with a list it sits above will go stale again. The preamble now says what it is and why it carries no number.

The missing deviation is recorded as a fourth bullet: step 1 named `./db/schema` among the exports and told the builder to write the tables under `src/db/schema/auth-schema/`, with Files / areas listing that directory and `src/db/index.ts`. What exists is one file, `packages/shared/src/db/schema.ts`, no barrel, no `db/index.ts`, and a third export of `./validation` that step 1 never mentions. Confirmed against the tree rather than assumed: `src/` holds exactly three files. The bullet points at the argument already written at `schema.ts:19-23` instead of restating it.

Closed 2026-09-23 by the third independent review (fresh subagent). The preamble at `current-feature.md:7-37` carries no count and says why, and its fourth bullet names the one-file schema and the `./validation` export. Checked against the tree: `packages/shared/src` holds `db/schema.ts`, `config/plan-limits.ts` and `validation/auth.ts`, and the three exports in `package.json` point at exactly those.

1/F-11 [P3] closed - `.env.example` claims every variable it lists is read today, and two are not

File: `.env.example:8` **Found:** 2026-09-23 by /audit (scope: current; lens: quality) **Why it matters:** Line 8 states "Every variable listed here is read by code that exists today", and justifies it: "A variable in an example file that nothing reads teaches the next person to configure something that does nothing". Two of the entries then say the opposite about themselves. `WIDGET_ORIGINS` at line 56 says "Arrives with build-plan item 2; nothing reads it yet", and `RESEND_API_KEY` at line 61 is claimed by item 6. Both are genuinely unread: nothing under `backend/src` references either name.

Each entry is honest on its own, which is why this is small. The header is not, and it is the line a reader trusts before reading the rest.

Suggested fix: One clause on line 8: every variable is either read today or says which build-plan item claims it. That keeps the rule the header exists to state while describing the file as it actually is.

Resolution: Fixed 2026-09-23 with the one clause the finding suggested. The header now says every variable is either read by code that exists today or names the build-plan item that claims it, and names the two of the second kind, `WIDGET_ORIGINS` and `RESEND_API_KEY`, so a reader meeting them further down is not surprised. The rule the header exists to state survives; it now describes the file as it actually is. The ImageKit, S3 and volume sentence is untouched, because that part was always true.

Closed 2026-09-23 by the third independent review (fresh subagent). `.env.example:8-16` now says every variable is read today or names the item that claims it, and names `WIDGET_ORIGINS` and `RESEND_API_KEY` as the two of the second kind. A search of `backend/src` still finds neither name, so the header now describes the file as it is. The two example database URLs carry placeholder credentials, not the real local ones.

Independent review

Status: passed **Target commit:** 9c798396f74abd62fd386065510a5acdcc7b9625 **Base commit:** 925920165bc7dc4dad188019dc1ad4a827ea9574 **Base ref:** main **Spec hash:** a30b8308b3a53bbd0698df68c3dce69854772094f8c4bf58328b9ca608b88095 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-09-23T18:56:59Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-09-23T19:02:40Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete

`925920165bc7dc4dad188019dc1ad4a827ea9574..9c798396f74abd62fd386065510a5acdcc7b9625` delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch.

Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

Commands

- `git rev-parse HEAD` , `git merge-base main HEAD` , `sha256sum blueprint/context/current-feature.md` , `git status --porcelain` : pass (target, merge base and spec hash match; only `review.md` differed)
- `tsc -p backend/tsconfig.json --noEmit` (TypeScript 7.0.2): pass
- `tsc -p packages/shared/tsconfig.json --noEmit` : pass
- `tsc -p frontend/tsconfig.json --noEmit --incremental false` (TypeScript 5.9.3): pass
- `eslint .` in `frontend` (the `lint` script's command): pass
- `curl` probes against the running API on `http://localhost:3001` (`/health` , unauthenticated `/me` , CORS preflights from a foreign and the app origin, `send-verification-otp` and `sign-in/email-otp` for an unknown address, a foreign-origin POST, unauthenticated `organization/create` , `sign-up/email`): pass
- `npm run build --workspace=backend` : unavailable (writes `dist/` ; this reviewer may write only the two review files)
- `npm run build --workspace=frontend` : unavailable (writes `.next/` beside the user's running dev server)
- `npm run db:generate --workspace=@scheduleads-app/shared` : unavailable (may write migration files)
- Signed-in runtime probes of `/me` , the non-admin organization-create refusal and the plan flip: unavailable (login codes print only to the user's API console; the plan flip writes to the database)
- Unit tests: unavailable (no test command declared)

Evidence

- Freshness: `HEAD` = target, `main` merge base = recorded base, spec SHA-256 matches, working tree clean except `blueprint/context/review.md` .
- Reviewed the full code delta: `backend/src/{index,database}.ts` , `backend/src/lib/{auth,active-organization,plan-gate,send-login-code}.ts` , `packages/shared` (package, tsconfig, drizzle config, schema, migration, journal, plan-limits, validation, seed), `frontend/app/{page,layout,sign-in/page,create-organization/page}.tsx` , `frontend/components/auth-card.tsx` , `frontend/lib/{api,auth-client}.ts` , `.env.example` , `.claude/launch.json` , package manifests and lockfile delta, and the standards, spec and skill edits. Static `prototypes/` , reference screenshots and the proposal documents were skimmed only; they contain no executable project code.
- Tenant boundary: `organizationId` comes only from the session (`active-organization.ts:101-141`), the member lookup filters on user and organization, and two or more memberships with no

active choice return nothing; `/me` returns only that organization's fields.

- Signup closed, re-proved live: unknown address gets `{"success":true}` with nothing sent and `INVALID_OTP` on verify; `sign-up/email` refused; foreign-origin POST refused `INVALID_ORIGIN`; foreign-origin preflight gets no `Access-Control-Allow-Origin`.
- Organization creation restricted to `user.role === "admin"`, confirmed against better-auth 1.7.5 `crud-org.mjs:47-58`.
- `plan` cannot be written from a request on create or update: `toZodSchema` drops `input: false` fields client-side and better-call keeps only the parsed body (`validator.mjs:17`).
- Secret scan of the added lines found no real credential; the two example database URLs in `.env.example` carry placeholder passwords.
- Performance: `/me` costs one session read, one member read (two on the fallback), one organization read. No unbounded work found.
- Tests: no runner is configured and the spec claims no automated coverage; no skipped, focused or placeholder tests exist.

Findings

- F-12 [P2] open: comments still describe open signup, self-serve business creation and server-side validation
- F-13 [P3] open: a signed-in user with no business is sent to a create form that can only refuse them
- F-14 [P3] open: sign-in and create-business forms bypass the form standard without saying so
- F-15 [P3] open: `coding-standards.md` still says migrations run from `backend`
- F-16 [P3] open: Better Auth endpoints already open two paths the spec reserves for later items
- Closed this pass: F-03, F-05, F-06, F-07, F-08, F-09, F-10, F-11
- No P0 or P1 finding is `open` or `fixed`.

Remaining risk

- Both workspace builds were not run by this reviewer (typecheck and lint passed instead); `next build` also runs Next's own checks that `tsc --noEmit` does not.
- `db:generate` was not re-run; the snapshot and schema are unchanged since the second review saw it report no changes.
- No signed-in runtime evidence from this reviewer: `/me` for a real session, the non-admin 403 on organization create and the plan flip rest on code reading plus Frank's recorded walkthrough.
- No unit test runner: the plan-limits resolver and the active-organization fallback have no automated coverage.
- Production cookie and deploy settings (`SameSite=None; Secure; Partitioned`, `COOKIE_DOMAIN`, platform Root Directory builds reaching `packages/shared`) and whether the live Railway database matches `schema.ts` are untested and belong to `/release`.